Formulación y búsqueda no informada en el problema de los misioneros y caníbales

Jair Páez

Escuela Politécnica Nacional, Quito, Ecuador
Asignatura: Inteligencia Artificial, Tarea DG_S4
jair.paez@epn.edu.ec

Resumen Se modela el problema clásico de los misioneros y caníbales como un espacio de estados con restricciones de seguridad y capacidad de transporte. Se implementan en Python los algoritmos de búsqueda primero en anchura (BFS) y búsqueda primero en profundidad (DFS), con estructuras FIFO y LIFO, respectivamente. En una ejecución del programa desarrollado, BFS registró 15 nodos expandidos y DFS registró 12. Ambos algoritmos encontraron soluciones válidas de 11 cruces. Se contrastan los resultados medidos con las propiedades teóricas de optimalidad y complejidad de ambos métodos.

Keywords: Inteligencia artificial · Espacio de estados · BFS · DFS · Búsqueda no informada

1. Introducción

La búsqueda no informada explora un problema sin disponer de una función heurística que estime la cercanía de un estado a la meta. Su aplicación requiere definir una representación de estados, las acciones legales y una condición de terminación [1,2]. El objetivo de este trabajo es formular matemáticamente el problema de los misioneros y caníbales, implementar BFS y DFS y comparar los resultados que reporta el programa mediante nodos expandidos, tiempo de ejecución y longitud del camino.

En el escenario considerado, tres misioneros y tres caníbales deben cruzar de la orilla izquierda a la derecha. El bote transporta como máximo dos personas y no puede viajar vacío. En una orilla con al menos un misionero, el número de caníbales no debe superar al de misioneros.

2. Formulación matemática

El problema se describe mediante la tupla

$$\mathcal{P} = (S, s_0, A, T, G, c), \tag{1}$$

donde S es el conjunto de estados admisibles, s_0 el estado inicial, A el conjunto de acciones disponibles, T la función de transición, G la prueba de meta y c el costo de cada movimiento [1].

2.1. Representación de estados y restricciones

Cada estado se representa mediante la terna

$$s = (m, c_i, b), \tag{2}$$

donde m indica los misioneros en la orilla izquierda, c_i los caníbales en esa misma orilla y $b \in \{0, 1\}$ la ubicación del bote. El valor $b = 1$ corresponde a la izquierda y $b = 0$ a la derecha. Los ocupantes de la orilla derecha se calculan por diferencia:

$$m_D = 3 - m, \qquad c_D = 3 - c_i. \tag{3}$$

El conjunto de estados seguros es

$$\begin{aligned} S = \{(m, c_i, b) \in \{0, 1, 2, 3\}^2 \times \{0, 1\} \mid \\ (m = 0 \ \vee \ m \geq c_i) \wedge \\ (3 - m = 0 \ \vee \ 3 - m \geq 3 - c_i)\}. \end{aligned} \tag{4}$$

La condición lógica permite orillas con solo caníbales, pero impide que estos superen en número a los misioneros cuando hay alguno presente.

2.2. Estado inicial y estado objetivo

Al comienzo, todas las personas y el bote se encuentran a la izquierda. La meta requiere que todos estén a la derecha:

$$s_0 = (3, 3, 1), \qquad s_g = (0, 0, 0). \tag{5}$$

La prueba de meta se define mediante

$$G(s) = \begin{cases} \text{verdadero}, & s = (0, 0, 0), \\ \text{falso}, & \text{en otro caso.} \end{cases} \tag{6}$$

2.3. Acciones y modelo de transición

Una acción $a = (x, y)$ traslada x misioneros y y caníbales. El conjunto de combinaciones de pasajeros es

$$A_0 = \{(1, 0), (0, 1), (2, 0), (0, 2), (1, 1)\}, \quad 1 \leq x + y \leq 2. \tag{7}$$

La acción debe contar con suficientes pasajeros en la orilla de partida. Esta restricción se representa mediante

$$D(s, a) = \begin{cases} x \leq m \ \wedge \ y \leq c_i, & b = 1, \\ x \leq 3 - m \ \wedge \ y \leq 3 - c_i, & b = 0. \end{cases} \tag{8}$$

Para una acción disponible, el estado candidato viene dado por

$$T(s, a) = \begin{cases} (m - x, c_i - y, 0), & b = 1, \\ (m + x, c_i + y, 1), & b = 0. \end{cases} \tag{9}$$

Solo son legales las acciones cuyo estado resultante cumple todas las restricciones del problema:

$$\text{Actions}(s) = \{a \in A_0 \mid D(s,a) \wedge T(s,a) \in S\}. \tag{10}$$

Por ejemplo, trasladar un misionero y un caníbal desde el inicio genera la transición $(3,3,1) \to (2,2,0)$, cuyo estado final es seguro.

2.4. Costo del camino

Como cada cruce tiene un costo unitario, se utiliza

$$c(s,a,s') = 1, \qquad C(\pi) = \sum_{i=1}^{n} c(s_{i-1}, a_i, s_i) = n. \tag{11}$$

Minimizar el costo total equivale a minimizar el número de cruces.

3. Implementación en Python

El programa creado representa estados y acciones mediante tuplas de enteros. La función `esEstadoValido` comprueba las restricciones de ambas orillas. La función `aplicarAccion` determina el nuevo estado y rechaza movimientos ilegales, mientras que `generarSucesores` enumera los estados alcanzables mediante las cinco combinaciones de pasajeros.

La implementación de BFS utiliza `collections.deque` como cola FIFO y la de DFS emplea una lista como pila LIFO. Ambos algoritmos registran los estados visitados para evitar ciclos y conservan las secuencias de acciones que permiten reconstruir el camino hasta la meta. El orden de las acciones es fijo y puede influir en el recorrido seleccionado por DFS.

4. Resultados y análisis

4.1. Protocolo de medición

Las métricas se obtuvieron mediante los mecanismos incorporados en el programa desarrollado. Cada algoritmo informa los nodos contabilizados como expandidos, la longitud del camino y el tiempo de su búsqueda. Este último se calcula en segundos con `time.perf_counter()`. La Tabla 1 reproduce los valores de la ejecución final mostrada en la consola, sin incorporar ensayos adicionales.

Cuadro 1. Métricas reportadas por la ejecución del programa.

Métrica	BFS	DFS
Nodos expandidos (contador del programa)	15	12
Tiempo de ejecución (s)	0.001339	0.000712
Longitud del camino (cruces)	11	11

Las dos estrategias encontraron soluciones válidas que alcanzan $(0, 0, 0)$ en 11 cruces. BFS procesó tres nodos más que DFS y el tiempo registrado fue mayor en esta ejecución. Como los tiempos son muy pequeños, estos valores describen la ejecución observada, sin establecer una ventaja temporal universal entre ambos algoritmos.

4.2. Secuencias de acciones encontradas

La Tabla 2 muestra las acciones de cada algoritmo en el orden registrado por el programa. M representa un misionero, C representa un caníbal y las flechas indican el sentido del cruce.

Cuadro 2. Rutas encontradas por BFS y DFS en 11 cruces.

Paso	Acción BFS	Estado BFS	Acción DFS	Estado DFS
0	Inicio	$(3, 3, 1)$	Inicio	$(3, 3, 1)$
1	$2C \rightarrow$	$(3, 1, 0)$	$M + C \rightarrow$	$(2, 2, 0)$
2	$\leftarrow C$	$(3, 2, 1)$	$\leftarrow M$	$(3, 2, 1)$
3	$2C \rightarrow$	$(3, 0, 0)$	$2C \rightarrow$	$(3, 0, 0)$
4	$\leftarrow C$	$(3, 1, 1)$	$\leftarrow C$	$(3, 1, 1)$
5	$2M \rightarrow$	$(1, 1, 0)$	$2M \rightarrow$	$(1, 1, 0)$
6	$\leftarrow M + C$	$(2, 2, 1)$	$\leftarrow M + C$	$(2, 2, 1)$
7	$2M \rightarrow$	$(0, 2, 0)$	$2M \rightarrow$	$(0, 2, 0)$
8	$\leftarrow C$	$(0, 3, 1)$	$\leftarrow C$	$(0, 3, 1)$
9	$2C \rightarrow$	$(0, 1, 0)$	$2C \rightarrow$	$(0, 1, 0)$
10	$\leftarrow M$	$(1, 1, 1)$	$\leftarrow C$	$(0, 2, 1)$
11	$M + C \rightarrow$	$(0, 0, 0)$	$2C \rightarrow$	$(0, 0, 0)$

Las soluciones siguen caminos distintos en los primeros y últimos movimientos. Esto refleja la diferencia en el orden de exploración de la frontera de búsqueda, aunque ambas rutas tienen la misma longitud.

5. Conclusión crítica

Las complejidades asintóticas difieren porque BFS explora los estados por niveles, con tiempo y espacio $O(b^d)$, mientras DFS profundiza en una rama, con tiempo $O(b^m)$ y espacio $O(bm)$ en el modelo teórico de búsqueda en árbol [2]. **DFS no garantiza el camino más corto** porque su pila LIFO prioriza la rama explorada más recientemente y el algoritmo se detiene al encontrar la primera meta, sin haber revisado necesariamente los caminos de menor profundidad. Por ello, podría hallar una ruta más larga antes que una más corta. En esta ejecución coincidió con BFS en 11 cruces debido al orden de exploración. **BFS sí garantiza una ruta mínima** cuando todos los cruces cuestan lo mismo, pues explora primero los niveles menos profundos. Su mayor requerimiento *teórico* de memoria se debe a que conserva simultáneamente los nodos pendientes de distintos caminos en la frontera, mientras DFS da prioridad a una rama. La memoria no se midió directamente en el programa, por lo que no se afirma una diferencia empírica de consumo. Los 15 nodos contabilizados en BFS y los 12 en DFS reflejan el recorrido concreto de este caso.

6. Conclusión final

Se formuló el problema de los misioneros y caníbales mediante estados, acciones y transiciones seguras y se implementaron BFS y DFS en Python. Ambos algoritmos alcanzaron la meta con 11 cruces, lo que permitió validar el modelo y contrastar su comportamiento con los fundamentos de búsqueda no informada.

Disponibilidad del código. El archivo fuente `Paez_DG_S4.py` se encuentra publicado en el repositorio público de GitHub `https://github.com/Jair-paez/IA`.

Referencias

1. Galarza García, D.E.: Inteligencia Artificial. Capítulo 2: Agentes inteligentes y formulación de problemas. Material de clase, Escuela Politécnica Nacional (2026)
2. Galarza García, D.E.: Inteligencia Artificial. Capítulo 2: Búsqueda sin información. Material de clase, Escuela Politécnica Nacional (2026)